

[Week7] <동적 할당과 메모리 취약점>

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // 1. 정수 1개 크기(4바이트) 할당
    int *p1 = (int *)malloc(sizeof(int));
    // 2. 정수 5개 크기(20바이트) 배열 할당
    int *p2 = (int *)malloc(sizeof(int) * 5);

    // 값 저장
    *p1 = 100;
    for(int i = 0; i < 5; i++) {
        p2[i] = i * 10;
    }

    // 출력
    int stack_var = 0;
    printf("p1의 값: %d\n", *p1);
    printf("p2[3]의 값: %d\n", p2[3]);

    printf("\n[주소 비교]\n");
    printf("Heap 영역 주소 (p1) : %p\n", (void*)p1);
    printf("Heap 영역 주소 (p2) : %p\n", (void*)p2);
    printf("Stack 영역 주소      : %p\n", (void*)&stack_var);

    // 해제
    free(p1);
    free(p2);
    printf("\n메모리 해제 완료.\n");

    return 0;
}
```

malloc()실패 시 처리: 메모리가 부족하면 `malloc` 은 `NULL` 주소를 반환한다. 이때 `NULL` 포인터에 값을 쓰려고 하면 프로그램이 즉시 강제 종료(Segmentation Fault)된다. 따라서 반드시 `if (ptr == NULL)` 로 검사하여 에러 메시지를 띄우고 안전하게 프로그램을 종료시켜야 한다.

free() 하지 않으면 생기는 일 (Memory Leak): 빌린 메모리를 반납하지 않으면, 운영체제는 그 땅이 계속 '사용 중'이라고 판단한다. 이런 코드가 반복 실행되는 서버 프로그램의 경우, 언젠가 컴퓨터의 모든 RAM 용량을 다 써버리게 되고 결국 시스템 전체가 먹통이 되거나 다운된다.(OOM: Out Of Memory).

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *victim = (int *)malloc(sizeof(int));
    *victim = 777;
    printf("free 전 victim의 값: %d, 주소: %p\n", *victim,
        (void*)victim);

    free(victim);

    printf("\n--- free() 이후 ---\n");
    printf("free 후 읽어보기: %d\n", *victim);

    return 0;
}
```

실행 결과 관찰: `free` 이후에 접근해도 프로그램이 바로 크래시(강제 종료)되지 않고 **정상 실행되는 것처럼 보인다**. 때로는 이전 값(`777`)이 그대로 남아있기도 하고, 어떨 때는 쓰레기 값이 나온다.

GDB 관찰 결과: `free` 직후 GDB에서 `x/4wx victim` 으로 메모리를 까보면 `777` 이 있던 자리에 갑자기 웬 '다른 메모리 주소(fd pointer)'가 덮어씌워진 것을 확인할 수 있다. 이는 운영체제가 빈방들을 서로 연결해 관리하기 위해 메모리 앞부분을 다음 빈방의 주소로 덮어버리기 때문이다.

Dangling Pointer가 위험한 이유: 가리키는 대상의 메모리가 해제되어 의미 없는 땅이 되었음에도, 포인터가 여전히 그 주소를 기억하고 있는 상태를 뜻한다. 해제된 땅에 다른 중요한 데이터(예: 관리자 비밀번호)가 들어왔는데, 과거의 포인터로 그 값을 실수로 덮어써 버릴 수 있기 때문에 위험한다.